

Experiment 1

Aim

To design, develop, and test an advanced Decentralized Application (DApp) that integrates a Solidity smart contract with a React.js frontend, enabling users to record electronic-waste submissions and interact with a transparent blockchain-based recycling reward system.

Problem Statement

Electronic waste is one of the fastest-growing waste streams in the world. Items such as mobile phones, laptops, batteries, chargers, and circuit boards contain valuable materials, but they may also release harmful substances when they are disposed of incorrectly. Traditional e-waste collection and reward systems are generally managed by centralized organizations, making the process dependent on a single authority and potentially reducing transparency.

The E-WasteFi project proposes a blockchain-based decentralized solution where e-waste submissions can be recorded through a smart contract and users can receive reward credits after their submissions are verified by an authorized recycler.

The physical inspection of e-waste is outside the scope of this experiment. The project demonstrates how blockchain, smart contracts, and a web-based DApp can be integrated to create a transparent and traceable recycling reward mechanism.

Theory

Decentralized Application (DApp)

A Decentralized Application (DApp) is a software application that uses blockchain technology for executing and storing important application logic and data. Unlike traditional applications, where the backend is hosted on a centralized server, a DApp can use smart contracts deployed on a blockchain.

In this experiment, E-WasteFi is developed as a decentralized electronic-waste collection and rewards platform. The application records the category, quantity, weight, status, and reward of each e-waste submission. Authorized recyclers can verify or reject submissions, and verified submissions can receive a reward only once.

4. Technologies Used

Technology	Purpose
Solidity	Development of the e-waste smart contract
Hardhat	Compiling, deploying, and testing the contract locally
Ethers.js	Ethereum-compatible contract interaction in the blockchain toolchain
Mocha and Chai	Automated smart-contract testing
React.js	Development of the frontend interface
JavaScript	Frontend programming
Vite	React development and build environment
MetaMask	Wallet connection and transaction authorization for a live deployment
Browser local storage	Preserving the current frontend demonstration records

Blockchain

A blockchain is a distributed digital ledger that stores transactions in blocks. Each block is connected to the previous block using cryptographic techniques.

Important characteristics of blockchain include:

- Decentralization
- Transparency
- Immutability
- Security
- Traceability

For this experiment, Hardhat provides a local Ethereum-compatible development environment. It allows the smart contract to be compiled and tested without using real Ethereum or real cryptocurrency.

Smart Contracts

A smart contract is a program stored and executed on a blockchain. It automatically performs predefined operations when its functions are called.

E-WasteFi uses one main smart contract called EWasteFi. The contract manages e-waste submissions, authorized recyclers, submission status, and reward balances.

EWasteFi Contract

The EWasteFi contract maintains:

- Submission records
- A submission counter
- Authorized recycler addresses
- User reward balances
- User submission history

Each submission has one of three statuses:

1. Pending
2. Verified
3. Rejected

The contract owner can add or remove recyclers. A user can submit supported electronic-waste categories with a positive quantity and weight. A recycler can then verify or reject a pending submission.

E-Waste Reward Mechanism

E-WasteFi uses a category-based reward mechanism. The reward rate for each item is:

E-waste category	Reward per item
Mobile Phone	25
Laptop	50
Battery	15
Charger	10
Circuit Board	20
Other	5

When a user submits e-waste, the reward is calculated as:

```
Reward = Quantity x Category Reward Rate
```

For example:

```
Category = Laptop  
Quantity = 2  
Reward Rate = 50 units per laptop  
Reward = 2 x 50 = 100 units
```

The reward is stored in the submission record. It is added to the user's reward balance only after an authorized recycler verifies the submission and calls the reward function.

E-Waste Verification

In a real-world system, electronic-waste verification could potentially use:

- Recycler inspection
- Collection-center records
- Digital weighing systems
- Product serial numbers
- Photographic evidence
- Authorized auditors
- QR-code or barcode tracking

Physical verification is not implemented in this experiment. Instead, verification is simulated by the authorized recycler calling `verifySubmission()`. A recycler may also call `rejectSubmission()` when a submission does not meet the collection requirements.

This keeps the experiment focused on demonstrating blockchain storage, access control, state transitions, and DApp integration.

React.js Frontend

React.js is used to create the graphical user interface of the E-WasteFi DApp. The frontend provides users with information such as:

- Connected wallet state
- E-waste category
- Quantity and weight
- Estimated reward
- Submission status
- Verified recycling weight
- Rewards earned
- Submission activity history

The interface contains Dashboard, Recycle, Rewards, and Activity views. Users can enter e-waste details through the recycle form, while recycler actions can update pending submissions.

The current frontend uses demo transaction responses and browser local storage so that the interface can be tested independently. The contract itself is implemented and tested in the Hardhat project.

MetaMask

MetaMask is a cryptocurrency wallet that allows web applications to communicate with blockchain networks.

In a live E-WasteFi deployment, MetaMask would:

1. Connect the user's wallet to the DApp.
2. Provide the user's blockchain address.
3. Connect to the selected local or public Ethereum-compatible network.
4. Request permission for blockchain transactions.
5. Sign transactions before sending them to the blockchain.

The user's private key is not directly handled by the React application.

Ethers.js

Ethers.js is a JavaScript library that allows web applications to interact with Ethereum-compatible blockchains.

In a live version of E-WasteFi, Ethers.js would be used to:

- Create a blockchain provider.
- Connect to MetaMask.
- Obtain wallet accounts.
- Create a contract instance from the address and ABI.
- Read submission and reward data.
- Send user and recycler transactions.
- Wait for transaction confirmations.

For example, the frontend can create a provider using:

```
const provider = new ethers.BrowserProvider(window.ethereum);
const signer = await provider.getSigner();
const contract = new ethers.Contract(contractAddress, contractAbi, signer);
```

ABI

ABI, or Application Binary Interface, defines how a frontend application communicates with a smart contract.

The frontend uses the ABI to know:

- Function names
- Function parameters
- Return values
- Whether a function is read-only or transactional

For example, `getSubmissionCount()` can be used to read the number of submissions. `getRewardBalance(address)` can be used to read a user's reward balance. `submitEwaste()` and `verifySubmission()` modify blockchain state and require transactions.

Read and Write Operations

Blockchain interactions can generally be divided into two types.

Read Operations: Read operations do not change blockchain state and normally do not require a transaction. Examples include:

```
getSubmission(id)
getUserSubmissionIds(address)
getRewardBalance(address)
getSubmissionCount()
```

Write Operations: Write operations modify blockchain state and require a transaction. Examples include:

```
submitEwaste(category, quantity, weight)
verifySubmission(id)
rejectSubmission(id)
issueReward(id)
```

MetaMask would be used to authorize these transactions in a live deployment.

Example

Suppose a user submits:

```
Category = Mobile Phone
Quantity = 2
Weight = 0.6 kg
Reward Rate = 25 units per phone
```

Therefore:

```
Reward = 2 x 25 = 50 units
```

The contract records the submission as Pending. After an authorized recycler verifies it, the status changes to Verified. When the recycler issues the reward, the user's reward balance increases by 50 units. If the submission is rejected, no reward is credited.

Implementation

Part A: Smart Contract Development with Hardhat

The smart contract is stored in `blockchain/contracts/EWasteFi.sol`.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

contract EWasteFi {
    enum Status { Pending, Verified, Rejected }

    struct Submission {
        uint256 id;
        address user;
        string category;
        uint256 quantity;
        uint256 weight;
        uint256 reward;
        Status status;
        uint256 submittedAt;
        uint256 verifiedAt;
        bool rewardCredited;
    }

    address public owner;
    uint256 public submissionCounter;
    mapping(uint256 => Submission) public submissions;
    mapping(address => bool) public recyclers;
    mapping(address => uint256) public rewardBalances;
    mapping(address => uint256[]) private userSubmissionIds;

    constructor() {
        owner = msg.sender;
    }

    function submitEwaste(
        string calldata category,
        uint256 quantity,
        uint256 weight
    ) external returns (uint256) {
        require(quantity > 0, "Invalid quantity");
        require(weight > 0, "Invalid weight");

        uint256 reward = _calculateReward(category, quantity);
        submissionCounter += 1;

        submissions[submissionCounter] = Submission({
            id: submissionCounter,
            user: msg.sender,
            category: category,
            quantity: quantity,
            weight: weight,
            reward: reward,
            status: Status.Pending,
            submittedAt: block.timestamp,
            verifiedAt: 0,
            rewardCredited: false
        });

        userSubmissionIds[msg.sender].push(submissionCounter);
    }
}
```

```

        return submissionCounter;
    }

    function verifySubmission(uint256 submissionId) external onlyRecycler {
        Submission storage submission = submissions[submissionId];
        require(submission.id != 0, "Submission not found");
        require(submission.status == Status.Pending, "Submission not pending");

        submission.status = Status.Verified;
        submission.verifiedAt = block.timestamp;
    }

    function issueReward(uint256 submissionId) external onlyRecycler {
        Submission storage submission = submissions[submissionId];
        require(submission.status == Status.Verified, "Not verified");
        require(!submission.rewardCredited, "Reward already credited");

        rewardBalances[submission.user] += submission.reward;
        submission.rewardCredited = true;
    }
}

```

The complete contract also includes custom errors, event declarations, recycler management, rejection handling, read functions, and the category reward calculation function.

Reward Calculation Function

The contract calculates the reward by comparing the submitted category with the supported categories:

```

function _calculateReward(
    string calldata category,
    uint256 quantity
) internal pure returns (uint256) {
    bytes32 categoryHash = keccak256(bytes(category));

    if (categoryHash == keccak256("Mobile Phone")) return quantity * 25;
    if (categoryHash == keccak256("Laptop")) return quantity * 50;
    if (categoryHash == keccak256("Battery")) return quantity * 15;
    if (categoryHash == keccak256("Charger")) return quantity * 10;
    if (categoryHash == keccak256("Circuit Board")) return quantity * 20;
    if (categoryHash == keccak256("Other")) return quantity * 5;

    revert UnsupportedCategory();
}

```

Access Control

The owner can manage recycler addresses using `addRecycler()` and `removeRecycler()`. Only addresses in the recycler mapping can verify, reject, or issue rewards. This prevents ordinary users from changing submission status or crediting rewards.

Part B: React Frontend Setup

The frontend is implemented in `frontend/src/App.jsx`. It provides a dashboard for submitting e-waste and viewing recycling activity.

The main submission calculation is equivalent to:

```
const selectedRate = rewardRates.find(
  (item) => item.name === formData.category
)?.reward || 0;

const estimatedReward = Number(formData.quantity) * selectedRate;
```

The frontend validates that the quantity is at least one and that the weight is greater than zero before creating a submission record. Pending submissions can be verified or rejected through the demonstration interface.

The current blockchain service is located in `frontend/src/services/blockchainService.js`:

```
let simulatedTransactionNumber = 1;

const createTransactionId = () => {
  const transactionNumber = String(simulatedTransactionNumber).padStart(6, "0");
  simulatedTransactionNumber += 1;
  return `SIM_TX_2026_${transactionNumber}`;
};

export const submitEwaste = () => ({
  success: true,
  simulated: true,
  transactionId: createTransactionId(),
});
```

This service currently simulates blockchain responses. To connect the UI to the deployed contract, the service would be replaced with Ethers.js provider, signer, ABI, contract address, and contract function calls.

Part C: Contract Testing

The tests are stored in `blockchain/test/EWasteFi.js`. They use Hardhat, Mocha, Chai, and multiple test accounts to check the contract behavior.

The test suite covers:

- Owner initialization
- Adding and removing recyclers
- Unauthorized owner and recycler actions
- Valid e-waste submissions
- Reward calculation for every supported category
- Pending, verified, and rejected statuses
- Reward issuance after verification
- Prevention of duplicate rewards
- Unsupported categories
- Zero quantity and zero weight validation
- Submission counter increments
- User submission history
- Reward balance updates

Example test:

```
await contract.connect(userOne).submitEwaste("Mobile Phone", 2, 500);
const submission = await contract.getSubmission(1);

expect(submission.category).to.equal("Mobile Phone");
expect(submission.quantity).to.equal(2n);
expect(submission.reward).to.equal(50n);
expect(Number(submission.status)).to.equal(0);
```

The expected result is a successful test suite. The current project test run completes with 24 passing tests.

Result

The E-WasteFi experiment successfully demonstrates a complete electronic-waste reward workflow. The smart contract records submissions, calculates category-based rewards, restricts recycler operations, manages status transitions, and prevents duplicate reward credits.

The React frontend presents the workflow through Dashboard, Recycle, Rewards, and Activity views. It allows users to enter e-waste information and view estimated rewards and submission statuses. The current frontend uses simulated transactions and local storage, while the Hardhat project provides the tested blockchain implementation.

Conclusion

The E-WasteFi experiment demonstrates the development and integration of a decentralized application using blockchain technology. The Solidity smart contract manages e-waste submissions, authorized recyclers, verification status, and reward balances, while Hardhat provides a local environment for compilation and testing.

The React.js frontend provides a practical web interface for submitting electronic waste and reviewing recycling activity. The project also demonstrates how Ethers.js and MetaMask can be integrated into the frontend for a live deployment through a contract ABI and deployed address.

The experiment shows how blockchain can help maintain transparent and traceable recycling records. Although physical e-waste verification and live token settlement are not implemented in the current version, the project provides a practical foundation for an incentive-based electronic-waste recycling network.

Experiment 1

Aim

To design, develop, and test **E-WasteFi**, a decentralized application for recording electronic-waste submissions, verifying them through authorized recyclers, and crediting transparent blockchain-based rewards. The project combines a Solidity smart contract deployed and tested with Hardhat and a React.js frontend that presents the recycling workflow to users.

Problem Statement

Electronic waste is one of the fastest-growing waste streams. Mobile phones, laptops, batteries, chargers, and circuit boards contain valuable recoverable materials, but they can also release harmful substances when they are disposed of incorrectly. Existing collection and reward programs are often managed by a centralized organization. This can make submission records difficult to audit and can reduce trust in the verification and reward process.

E-WasteFi proposes a blockchain-based record system in which:

- A user submits the category, quantity, and weight of electronic waste.
- The submission receives a unique identifier and a calculated reward amount.
- An authorized recycler verifies or rejects the submission.
- A verified submission can receive its reward only once.
- Submission events and reward credits can be traced through contract events.

The project demonstrates the technical workflow using local Hardhat tests and a React interface. Physical inspection of waste and real-world recycling logistics are outside the scope of this experiment.

Theory

Decentralized Application

A decentralized application, or DApp, is an application whose important logic is implemented by smart contracts on a blockchain. Instead of depending entirely on a private backend database, the application can use blockchain storage, access control, events, and transactions to maintain an auditable record.

Blockchain

A blockchain is a distributed ledger that stores transactions in cryptographically linked blocks. Its relevant characteristics for E-WasteFi are:

- **Transparency:** contract state and emitted events can be inspected.
- **Immutability:** confirmed records cannot be silently edited.
- **Traceability:** each submission and reward action has a history.
- **Access control:** only the contract owner can manage recyclers, and only approved recyclers can verify submissions or issue rewards.

Hardhat provides the local Ethereum-compatible development and testing environment used for this project. It allows the contract behavior to be tested without using real cryptocurrency or a public network.

Smart Contract

A smart contract is a program stored and executed by the blockchain. The EWasteFi contract stores each submission in a mapping and assigns it one of three statuses:

1. Pending
2. Verified
3. Rejected

The owner manages the recycler allowlist. A user can submit a supported waste category with a positive quantity and weight. An authorized recycler can then verify or reject the pending record. Rewards are stored in the rewardBalances mapping and are credited only after verification.

Reward Mechanism

The reward is calculated from the waste category and quantity:

Category	Reward per item
Mobile Phone	25
Laptop	50
Battery	15
Charger	10
Circuit Board	20
Other	5

The calculation is:

```
reward = quantity x category reward rate
```

The current contract records reward units in an internal balance mapping. It does not yet transfer an ERC-20 token or native cryptocurrency to the user.

Technologies Used

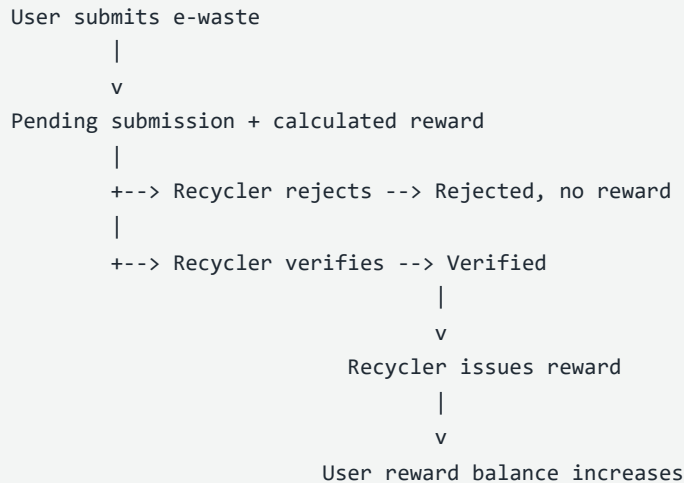
Technology	Purpose
Solidity 0.8.28	Writing the E-WasteFi smart contract
Hardhat 3	Local blockchain development, compilation, and testing
Ethers.js	Contract interaction in the blockchain toolchain
Mocha and Chai	Smart-contract behavior tests and assertions
React.js	Building the frontend interface
Vite	Frontend development and production build tool
JavaScript	Frontend application logic
MetaMask-compatible wallet flow	Intended wallet connection model for a deployed DApp
Browser local storage	Preserving the current frontend demo submissions

System Design

The system has three main roles:

- **User:** submits electronic waste and views submission history, status, and rewards.
- **Recycler:** verifies or rejects pending submissions and issues rewards for verified records.
- **Owner:** adds or removes authorized recycler addresses.

The main workflow is:



Smart Contract Implementation

The contract is located at `blockchain/contracts/EWasteFi.sol`.

Stored Data

Each Submission contains:

- Submission ID
- User address
- Waste category
- Quantity
- Weight
- Calculated reward
- Current status
- Submission timestamp
- Verification timestamp
- Whether the reward was credited

The contract also stores the owner address, the submission counter, approved recyclers, each user's reward balance, and each user's submission identifiers.

Main Functions

Function	Description	Permission
<code>addRecycler(address)</code>	Adds an address to the recycler allowlist	Owner
<code>removeRecycler(address)</code>	Removes an address from the recycler allowlist	Owner
<code>submitEwaste(category, quantity, weight)</code>	Creates a pending submission and calculates its reward	Any user
<code>verifySubmission(id)</code>	Changes a pending submission to verified	Recycler
<code>rejectSubmission(id)</code>	Changes a pending submission to rejected	Recycler
<code>issueReward(id)</code>	Credits a verified submission's reward balance	Recycler
<code>getSubmission(id)</code>	Returns one submission	Read-only
<code>getUserSubmissionIds(address)</code>	Returns a user's submission IDs	Read-only
<code>getRewardBalance(address)</code>	Returns a user's reward balance	Read-only
<code>getSubmissionCount()</code>	Returns the number of submissions	Read-only

Validation and Events

The contract rejects zero addresses, zero quantities, zero weights, unsupported categories, unknown submission IDs, non-pending submissions, non-verified submissions, duplicate reward credits, and unauthorized callers.

The following events provide an auditable activity trail:

- `EWasteSubmitted`
- `SubmissionVerified`
- `SubmissionRejected`
- `RewardIssued`
- `RecyclerAdded`
- `RecyclerRemoved`

React Frontend

The React frontend is located in `frontend/src/App.jsx`. It provides Dashboard, Recycle, Rewards, and Activity views. The recycle form accepts:

- E-waste category
- Quantity
- Weight in kilograms
- Optional description

The selected category and quantity are used to display an estimated reward before submission. The interface also displays verified recycling weight, earned rewards, submission statuses, recycler information, and activity history.

The current frontend is a demonstration layer. Its `blockchainService.js` creates simulated transaction identifiers for submission, verification, and reward actions, while local storage preserves demo records in the browser. The Hardhat contract and its test suite provide the actual blockchain implementation. For a production deployment, the service layer should be connected to `window.ethereum`, an Ethers.js provider, the deployed contract address, and the generated ABI.

A typical live connection would use the following pattern:

```
const provider = new ethers.BrowserProvider(window.ethereum);
const signer = await provider.getSigner();
const contract = new ethers.Contract(contractAddress, contractAbi, signer);
```

Read operations such as `getSubmissionCount()` do not change blockchain state. Write operations such as `submitEwaste()` and `verifySubmission()` require a signed transaction and confirmation from the connected wallet.

Testing

The contract tests are located in `blockchain/test/EWasteFi.js`. They cover:

- Owner initialization and recycler management
- Permission failures for non-owner and non-recycler accounts
- Valid submissions and reward calculations
- Pending, verified, and rejected status transitions
- Reward issuance after verification
- Prevention of duplicate reward issuance
- Unsupported categories and invalid quantity or weight
- Submission counter and user submission history
- Reward balance updates

Representative example:

```
await contract.connect(userOne).submitEwaste("Mobile Phone", 2, 500);
const submission = await contract.getSubmission(1);

expect(submission.reward).to.equal(50n);
expect(Number(submission.status)).to.equal(0);
```


The expected result is a successful test suite in the Hardhat project, with a mobile-phone quantity of two producing a reward of 50 units and an initial Pending status.

Result

The E-WasteFi experiment demonstrates a complete e-waste reward workflow. Solidity contract logic validates submissions, controls recycler permissions, records status changes, calculates category-based rewards, and prevents duplicate credits. The React interface presents this workflow in a usable dashboard and records demo activity for frontend testing.

The project therefore shows how blockchain can improve the transparency and traceability of an electronic-waste incentive program. The current implementation is suitable for local experimentation. A production version would require deployment to a selected network, a real wallet connection, contract address and ABI configuration, a token or payment settlement mechanism, and an off-chain process for physically validating collected waste.

Conclusion

The E-WasteFi experiment successfully demonstrates the design of a decentralized application for electronic-waste collection and rewards. The E-WasteFi smart contract maintains submission records, enforces owner and recycler permissions, calculates rewards for supported categories, and credits verified rewards only once. Hardhat tests verify the important success and failure paths.

The React frontend gives users a clear way to submit e-waste and review statuses, weights, and rewards. Although the current browser service simulates transaction responses, its structure corresponds to the contract workflow and can be connected to Ethers.js and MetaMask for a live deployment. The project provides a practical foundation for a transparent, traceable, and incentive-based e-waste recycling network.